

1.오늘의 핵심 목표

오늘의 핵심 목표: DB/MCP 구현이 아니라 Agent Tool 설계

3일차는 PostgreSQL이나 FastMCP 문법을 깊게 배우는 시간이 아니라, Agent Tool 설계 관점에서 내부 시스템 연동 구조를 바라보는 시간입니다. 여기서 Tool은 개발자가 사용하는 일반적인 도구가 아니라, Agent가 업무를 수행하기 위해 호출할 수 있는 기능 단위입니다.

예를 들어 설비 상태 조회, 알람 이력 확인, 품질 지표 조회, 기술 문서 검색 같은 기능을 Agent Tool로 볼 수 있습니다. 제조 현업에는 DB, Log, 기술 문서, 품질 지표 등 다양한 업무 정보가 존재합니다. Agent가 이 정보를 직접 다루도록 만들면 구조가 복잡해지고, 권한·오류·응답 형식·추적 기준을 통제하기 어려워집니다.

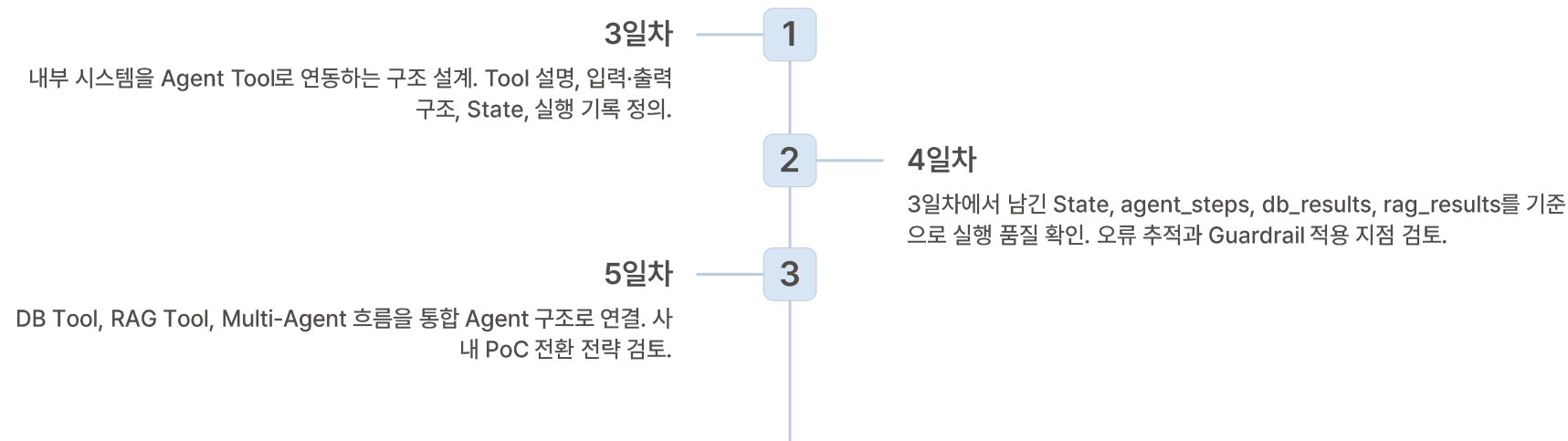
따라서 각 정보 접근 기능을 Agent가 호출 가능한 Tool로 감싸고, Tool 이름, 입력값, 출력값, 오류 형식을 명확히 설계해야 합니다. 오늘은 Tool 분리, Tool Contract, LangGraph Routing, Trace, Multi-Agent 역할 분담을 중심으로 내부 시스템을 Agent 구조에 연결하는 기준을 정리합니다.

 PostgreSQL과 FastMCP는 학습의 중심이 아니라, 실제 내부 시스템과 MCP Tool 구조를 재현하기 위한 교육용 수단입니다. 핵심은 코드를 실행하는 것이 아니라, 현업 적용 시 어디를 어떻게 바꿔야 하는지 판단하는 것입니다.

남은 3일 과정의 방향

3일차는 제조 현업의 내부 시스템을 Agent Tool로 연동하기 위한 구조를 설계하는 날입니다. DB 조회, 로그 확인, 기술 문서 검색은 각각 독립된 기능처럼 보이지만, Agent 아키텍처에서는 LLM이 어떤 기능을 선택하고, 어떤 입력을 전달하며, 어떤 결과를 다음 단계의 판단 근거로 사용할지를 함께 설계해야 합니다.

이때 3일차에서 정의하는 Tool 설명, 입력·출력 구조, State, 실행 기록은 이후 평가와 통합의 기준선이 됩니다. 3일차 산출물은 단독 실습 결과가 아니라 4일차 평가와 5일차 통합으로 이어지는 연결 자산입니다.



Trace는 단순 로그가 아니라 Tool 선택이 적절했는지, 입력과 출력이 예상 형식을 따랐는지, 위험한 응답이나 부적절한 실행이 있었는지를 판단하는 근거가 됩니다. 교육용 예제에서 사용한 DB 접속 방식, 문서 저장소, 권한 구조, 보안 정책, 감사 로그 기준은 실제 현업 구조로 대체할 때 반드시 검토해야 할 전환 대상입니다.

오늘 다룰 핵심 설계 질문

오늘 수업은 기능을 하나씩 실행하는 순서가 아니라, 제조 AI Agent를 설계할 때 반드시 답해야 하는 질문을 따라가는 흐름입니다. 각 질문은 설계 판단의 기준이 되며, 실습 파일과 직접 연결됩니다.

1

Tool 단위 분할 기준

제조 DB, Log, 기술 문서 검색 기능을 어떤 Tool 단위로 나눌 것인가? Tool이 너무 크면 역할이 모호해져 LLM이 잘못 선택할 수 있고, 너무 작으면 호출 수가 많아져 흐름이 복잡해질 수 있습니다. 업무 단위, 데이터 소스, 권한 수준, 위험도를 함께 고려해 Tool 경계를 정해야 합니다.

2

Tool Contract 설계

LLM이 Tool을 잘 선택하도록 Tool 이름, 설명, 입력·출력·오류 처리 기준을 어떻게 설계할 것인가? 이는 Agent가 사용할 수 있는 업무 기능을 표준화하는 과정입니다. Tool 이름만으로도 LLM이 언제 이 Tool을 써야 하는지 판단할 수 있어야 합니다.

3

실행 기록과 평가 연결

실행 이력을 State, agent_steps, db_results, rag_results, 결과 파일 형태로 어떻게 남기고, 이를 4일차 평가와 Guardrail 검증으로 어떻게 연결할 것인가? 실행 기록은 단순 로그가 아니라 설계 판단의 근거입니다.

 이 세 가지 질문은 오늘 수업 전체를 관통하는 설계 기준입니다. 각 실습 파일을 읽을 때도 이 질문을 기준으로 해석하는 것이 중요합니다.

1장 정리: 3일차 수업을 바라보는 기준

이 장은 3일차 수업을 어떤 관점으로 바라볼지 정리하는 도입 장입니다. 3일차를 DB 연결, FastMCP 문법, 코드 실행 중심으로 보면 오늘 수업의 핵심을 놓치지 쉽습니다. 오늘은 설계 관점에서 내부 시스템을 Agent Tool로 감싸는 구조를 봅니다.

제조 현업 시스템을 어떤 Tool 단위로 나누고, Tool Contract와 LangGraph Routing, Multi-Agent State 전달, Trace 구조로 어떻게 연결할지 판단하는 것이 핵심입니다.

설계 관점으로 읽는 실습 파일

postgres_db_tool.py, search_manual_tool.py,
manufacturing_mcp_client.py,
manufacturing_mcp_server.py, multi_agent_roles.py,
day3_mcp_tool_agent_v2.py 같은 파일을 설계 관점으로 읽습니다.

실행 기록 관점으로 확인하는 항목

State, agent_steps, db_results, rag_results, 결과
JSON/Markdown을 실행 기록 관점으로 확인합니다. 이 기록들은 4일
차 평가의 입력 자료가 됩니다.

Agent Tool은 단순 함수가 아니라 Agent가 업무를 수행하기 위해 호출하는 표준화된 업무 기능입니다. 따라서 파일의 코드 내용만 보는 것이 아니라, 이 Tool이 어떤 업무 기능을 대표하는지, LLM이 언제 선택해야 하는지, 입력과 출력은 어떤 기준으로 제한되는지, 실행 결과는 어떤 형태의 기록으로 남는지를 함께 확인해야 합니다.

5일 과정 전체 흐름

5일 과정은 각각 분리된 주제를 순서대로 나열하는 방식이 아니라, 하나의 제조 AI Agent 아키텍처를 단계적으로 완성하는 흐름으로 구성됩니다. 각 일차는 이전 일차의 결과물을 기반으로 구조를 확장합니다.



1일차: Agent 기본 흐름

Agent가 사용자 요청을 받아 의도를 파악하고 응답을 생성하는 기본 흐름을 확인합니다.



2일차: RAG와 State

기술 문서 근거와 작업 상태가 Agent 흐름 안에서 어떻게 유지되는지 살펴봅니다.



3일차: 내부 시스템 연동 (오늘)

제조 DB, Log, 기술 문서 검색 기능을 Agent가 호출 가능한 MCP Tool로 정리하고, Tool 이름, 입력, 출력, 오류 형식을 설계합니다.



4일차: 실행 품질 평가

3일차에서 남긴 State와 실행 기록을 기준으로 실행 품질과 Guardrail 적용 지점을 평가합니다.



5일차: 통합 및 PoC 전환

DB Tool, RAG Tool, Multi-Agent 흐름을 통합 Agent 구조로 연결하여 사내 PoC 전환 전략을 검토합니다.

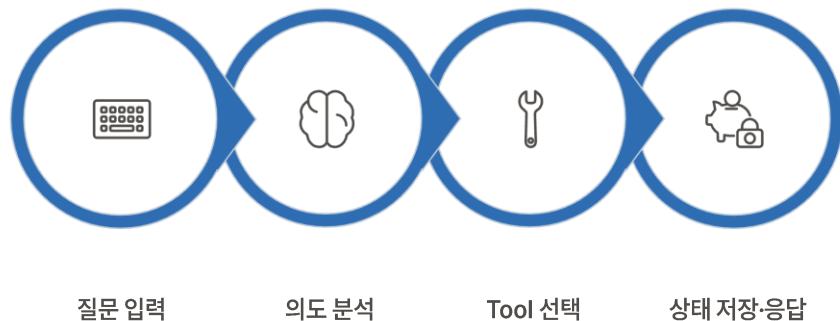


3일차는 앞선 실습 결과를 실제 제조 업무 시스템과 연결 가능한 구조로 확장하는 핵심 구간입니다. 오늘의 설계 결정이 4일차와 5일차의 품질을 결정합니다.

2.3일차 아키텍처 위치

3일차 아키텍처 위치

3일차는 사용자 요청이 실제 Tool 호출로 연결되는 구조를 확인하는 날입니다. 사용자가 설비 상태, 반복 알람, 품질 영향, 조치 절차와 관련된 질문을 입력하면 CoordinatorAgent는 요청의 의도와 필요한 정보를 정리합니다.



이후 Agent는 질문의 성격에 따라 DB/Log Tool 또는 RAG Tool 중 필요한 Tool을 선택해 호출합니다. 여기서 Tool은 개발자가 사용하는 일반적인 도구가 아니라, **Agent가 업무를 수행하기 위해 호출할 수 있는 기능 단위**입니다. 예를 들어 설비 상태 조회, 알람 이력 확인, 품질 지표 조회, 기술 문서 검색 같은 기능을 Agent Tool로 볼 수 있습니다. Tool 호출 결과는 State에 저장되고, 다음 Agent 또는 Summary/Action 생성 단계로 전달됩니다. 이때 중요한 것은 최종 답변만 생성하는 것이 아니라 어떤 Tool이 호출되었고, 어떤 입력과 출력이 오갔으며, 실패나 예외가 어떻게 처리되었는지를 State와 agent_steps, db_results, rag_results, 결과 파일로 확인할 수 있게 만드는 것입니다.



이 구조가 있어야 4일차의 실행 품질 평가와 Guardrail 검증이 가능합니다. 실행 기록 없이는 평가도, 개선도 불가능합니다.

전체 아키텍처에서 LangGraph가 맡는 역할

LangGraph는 단순히 복잡한 코드를 구성하기 위한 도구가 아니라, Agent가 어떤 조건에서 어떤 단계로 이동할지 설계하는 흐름 제어 구조입니다. MCP가 Agent가 사용할 Tool의 규격과 연결 방식을 정리한다면, LangGraph는 그 Tool을 어떤 조건과 순서로 호출할지 결정하는 역할을 합니다.

LangGraph의 Node 구성

요청 분석, Tool 선택, DB 조회, RAG 검색, 결과 종합, 검증 단계를 각 Node로 볼 수 있습니다. 각 Node는 명확한 입력과 출력을 가지며, State를 통해 다음 Node로 정보를 전달합니다.

Edge와 Routing 설계

질문의 성격이나 실행 결과에 따라 다음 단계가 달라지는 흐름을 Edge와 Routing으로 설계합니다. 설비 이력이나 품질 수치가 필요한 질문은 DB/Log Tool로, 조치 절차나 매뉴얼 근거가 필요한 질문은 RAG Tool로 보냅니다.

정보가 부족하면 추가 질문 단계로 이동하고, 위험 요청이나 과도한 조치는 Guardrail 또는 Human Review 흐름으로 보낼 수 있어야 합니다. 이 과정에서 State는 단계 간 작업 정보를 전달하는 기준이 되고, agent_steps와 결과 파일은 실행 흐름을 확인하는 근거가 됩니다.

- ❏ 3일차에서는 LangGraph 문법 자체보다 Tool 호출 조건과 State 전달 흐름을 설계하는 관점에 집중합니다. 문법은 수단이고, 설계 판단이 목적입니다.

3일차 실습 예제 파일이 맡는 역할

3일차 실습 파일은 단순히 실행할 Python 파일 목록이 아니라, 전체 Agent 아키텍처의 각 계층을 보여주는 예제입니다. 각 파일이 어떤 계층을 담당하는지 이해하는 것이 코드를 따라 실행하는 것보다 중요합니다.

파일명	역할 및 설계 관점
postgres_db_tool.py	교육용 제조 DB와 Log 조회 기능을 Agent Tool로 감싸는 계층. 설비 상태, 알람 이력, 품질 지표 조회 기능을 Tool 단위로 표준화합니다.
search_manual_tool.py	2일차에서 다룬 RAG 검색을 Agent가 호출 가능한 Tool로 표준화하는 역할. 기술 문서 검색 기능의 입력·출력 구조를 정의합니다.
manufacturing_mcp_client.py	TOOL_DESCRIPTIONS를 통해 Tool 이름과 설명을 확인하는 기준이 됩니다. LLM이 Tool을 선택할 때 참조하는 메타데이터 구조를 보여줍니다.
manufacturing_mcp_server.py	@mcp.tool 등록 구조를 통해 Python 함수를 MCP Tool로 공개하는 방식을 보여줍니다.
tool_registry_fallback.py	MCP 실행이 어려운 환경에서 get_equipment_overview와 search_manual 중심으로 예비 호출 구조를 제공하는 단순 fallback 예제입니다.
multi_agent_roles.py	어떤 Agent가 어떤 Tool을 호출하고 어떤 State를 넘기는지 확인하는 파일. Multi-Agent 역할 분담 구조를 보여줍니다.
day3_mcp_tool_agent_v2.py	3일차 전체 흐름을 실행하고 결과 JSON/Markdown을 저장합니다. State의 agent_steps, db_results, rag_results는 4일차 평가와 오류 분석의 입력 자료로 활용됩니다.

 실습 파일은 코드를 따라 실행하는 대상이 아니라 아키텍처 계층을 읽는 기준으로 보아야 합니다. 각 파일이 전체 구조에서 어떤 위치를 차지하는지 파악하는 것이 핵심입니다.

교육용 구조와 현업 구조의 차이

수업에서는 실제 사내 시스템을 직접 사용하지 않고 교육용 구조로 제조 AI Agent 아키텍처를 재현합니다. 이 구조는 현업 시스템을 단순화한 축소판이므로, 그대로 운영 환경에 적용하는 것이 목적이 아닙니다.

교육용 구조

교육용 PostgreSQL → 실제 제조 DB 대체
Markdown 문서 → 실제 기술 문서 저장소 대체
fallback Tool Registry → MCP 실행 어려울 때 예비 호출 구조
State/agent_steps → 운영 로그·감사 로그 이해용 연습 자료

현업 전환 시 검토 항목

실제 DB 접속 방식 및 연결 보안
사내 문서 저장소 및 접근 권한
데이터 접근 권한 및 승인 절차
보안 정책 및 감사 로그 기준

교육용 Guardrail은 사내 보안 정책과 개인정보·기밀정보 처리 기준으로 확장되어야 하며, 교육용 Agent 역할은 실제 업무 조직, 권한, 승인 흐름과 연결되어야 합니다.

따라서 중요한 것은 교육용 파일을 그대로 사용하는 것이 아니라, 어떤 계층을 현업 구조로 대체해야 하는지 판단하는 것입니다. 이 판단 기준이 5일차 사내 PoC 전환 전략의 출발점이 됩니다.



현업 적용 시에는 반드시 DB 접속 방식, 문서 저장소, 데이터 접근 권한, 승인 절차, 보안 정책, 감사 로그 기준을 실제 환경에 맞게 바꿔야 합니다. 교육용 구조를 그대로 운영 환경에 배포하지 않도록 주의하십시오.

2장 정리: 전체 아키텍처에서 3일차가 맡는 역할

이 장은 3일차가 전체 제로 AI Agent 아키텍처에서 어떤 위치를 차지하는지 정리하는 장입니다. 3일차는 1일차의 Agent 기본 흐름과 2일차의 RAG/State 구조를 실제 내부 시스템 연동 구조로 확장하는 단계입니다.

MCP Tool의 역할

Agent가 사용할 외부 기능의 규격을 정리합니다. Tool 이름, 설명, 입력·출력 구조, 오류 처리 기준을 표준화하여 LLM이 올바른 Tool을 선택할 수 있도록 합니다.

LangGraph의 역할

Tool을 어떤 조건과 순서로 호출할지 설계합니다. Node, Edge, Routing을 통해 Agent 흐름을 제어하고, 예외 상황에서의 분기를 처리합니다.

Multi-Agent 구조의 역할

단순히 역할 이름을 나누는 것이 아니라, Agent별 Tool 호출 책임과 State 전달 기준을 명확히 하는 구조입니다. 각 Agent가 어떤 Tool을 호출하고 어떤 결과를 다음 Agent에 전달하는지 정의합니다.

실행 기록의 역할

State와 agent_steps, db_results, rag_results, 결과 JSON/Markdown은 실행 흐름을 해석하는 자료이며, 4일차의 실행 품질 평가, 오류 추적, Guardrail 검증으로 이어지는 근거가 됩니다.

3일차 실습 예제 파일은 이러한 계층을 안전하게 학습하기 위한 교육용 구조이며, 현업 적용 시에는 실제 DB, 문서 저장소, 권한, 보안, 감사 로그 체계로 대체되어야 합니다. 다음 장에서는 Claude Code를 활용해 3일차 예제 구조를 이해하고, 설계 리뷰와 제한 수정 원칙을 확인합니다.

3일차 학습 체크리스트

오늘 수업을 마친 후 다음 항목을 스스로 점검해 보십시오. 각 항목은 단순한 확인이 아니라, 4일차 평가와 5일차 PoC 전환을 위한 설계 판단 기준입니다.

→ Tool 분할 기준을 설명할 수 있는가?

제조 DB 조회, 로그 확인, 기술 문서 검색을 각각 별도의 Tool로 나누는 이유를 업무 단위, 데이터 소스, 권한 수준, 위험도 관점에서 설명할 수 있어야 합니다.

→ Tool Contract의 구성 요소를 이해하는가?

Tool 이름, 설명, 입력값, 출력 구조, 오류 처리 기준, 검증 규칙이 왜 필요한지, 그리고 LLM이 Tool을 선택할 때 이 정보를 어떻게 활용하는지 설명할 수 있어야 합니다.

→ State와 실행 기록의 의미를 파악하는가?

agent_steps, db_results, rag_results가 단순 로그가 아니라 Tool 선택의 적절성, 입출력 형식 준수 여부, 위험 응답 여부를 판단하는 근거임을 이해해야 합니다.

→ 교육용 구조와 현업 구조의 차이를 구분하는가?

교육용 PostgreSQL, Markdown 문서, fallback Tool Registry가 현업에서 무엇으로 대체되어야 하는지, 그리고 어떤 보안·권한·감사 기준이 추가되어야 하는지 판단할 수 있어야 합니다.

- ✔ 이 네 가지 항목을 명확히 설명할 수 있다면, 3일차의 핵심 설계 관점을 충분히 이해한 것입니다. 4일차에서는 오늘 남긴 실행 기록을 기준으로 품질 평가와 Guardrail 검증을 진행합니다.